Cricket Ticket Booking System

Module: Python Programming • Practical Assignment Submission Report

Trainee Name:	Ameer Hamza	**Trainee ID:**	TRN-2026-001056
Programme:	Artificial Intelligence	**Stream:**	AI
Cohort:	PSEB-AI-LHR-01	**Status:**	Enrolled
Lead Trainer:	Bilal Nadeem	**Venue:**	Online

1. Assignment Overview & Objectives

This project implements an interactive, menu-driven Cricket Stadium Ticket Booking System in Python. The application simulates real-time stadium seat reservations while strictly adhering to fundamental programming principles without relying on complex classes, external packages, or dictionaries.

- **Core Data Structure:** Two-dimensional Python list holding seat number, category name, and booking status.
- **Search & Update Algorithm:** Linear loop traversal for seat verification and in-place status modification ('Available' -> 'Sold').
- **Integrity Checks:** Comprehensive validation preventing duplicate bookings and handling non-existent seat inputs.
- **Analytics & Metrics:** Real-time tracking of remaining tickets and dynamic computation of total tickets, sold seats, and percentage capacity sold.

2. Functional Tasks Breakdown

Task	Description	Implemented Behavior
Task 1: Display All Tickets	View complete stadium inventory	Traverses 2D list and displays Seat Number, Seat Name, and Status in aligned tabular format.
Task 2: Book a Ticket	Purchase ticket with validation	Searches seat by number, updates status in-place to 'Sold', prevents duplicate bookings, rejects invalid seats.
Task 3: Display Available	Filter available inventory	Iterates with conditional filter to display exclusively seats marked as 'Available'.
Task 4: Display Sold	Filter booked inventory	Iterates to display only seats marked as 'Sold' alongside their seat categories.
Task 5: Remaining Tickets	Capacity analytics (Bonus included)	Counts available vs. sold seats and calculates overall percentage of tickets sold.

3. Complete Python Source Code (.py)

Below is the complete, self-contained Python source code from ticket_booking_system.py. You may review or replace this section directly.

```
# ============================================================================
# Cricket Ticket Booking System
# Python Programming Assignment - 1
# ============================================================================

# 1. Initial Ticket Data (Stored in a 2D Python List)
# Each record contains: [Seat Number, Seat Name, Status]
tickets = [
    [101, "VIP", "Available"],
    [102, "VIP", "Available"],
    [103, "General", "Available"],
    [104, "General", "Available"],
    [105, "Premium", "Available"],
    [106, "Premium", "Available"],
    [107, "General", "Available"],
    [108, "VIP", "Available"]
]

# 2. Main Menu-Driven Program
while True:
    print("\n================================")
    print("      CRICKET TICKET SYSTEM      ")
    print("================================")
    print("1. Display All Tickets")
    print("2. Book a Ticket")
    print("3. Display Available Tickets")
    print("4. Display Sold Tickets")
    print("5. Remaining Tickets")
    print("6. Exit")

    choice = input("\nEnter your choice: ").strip()

    # Task 1: Display All Tickets
    if choice == "1":
        print("\n--- All Stadium Tickets ---")
        print(f"{'Seat Number':<15}{'Seat Name':<15}{'Status':<15}")
        print("-" * 45)
        for ticket in tickets:
            print(f"{ticket[0]:<15}{ticket[1]:<15}{ticket[2]:<15}")

    # Task 2: Book a Ticket
    elif choice == "2":
        seat_input = input("\nEnter seat number: ").strip()
```

```
        # Check if the user entered a valid integer
        if not seat_input.isdigit():
            print("Invalid input! Please enter a valid numeric seat number.")
        else:
            seat_number = int(seat_input)
            found = False

            # Linear search through the list
            for ticket in tickets:
                if ticket[0] == seat_number:
                    found = True
                    if ticket[2] == "Available":
                        ticket[2] = "Sold"  # Update status directly in the list
                        print("\nTicket Booked Successfully!")
                        print(f"Seat Number: {ticket[0]}")
                        print(f"Seat Name: {ticket[1]}")
                        print(f"Status: {ticket[2]}")
                    else:
                        print("\nSorry! This ticket has already been sold.")
                    break  # Stop search once match is found

            if not found:
                print(f"\nSeat number {seat_number} does not exist. Please check the
seat number.")

    # Task 3: Display Available Tickets
    elif choice == "3":
        print("\n--- Available Tickets ---")
        print(f"{'Seat Number':<15}{'Seat Name':<15}{'Status':<15}")
        print("-" * 45)
        available_found = False
        for ticket in tickets:
            if ticket[2] == "Available":
                print(f"{ticket[0]:<15}{ticket[1]:<15}{ticket[2]:<15}")
                available_found = True
        if not available_found:
            print("No tickets available. All seats are sold out!")

    # Task 4: Display Sold Tickets
    elif choice == "4":
        print("\n--- Sold Tickets ---")
        print(f"{'Seat Number':<15}{'Seat Category':<15}")
        print("-" * 30)
        sold_found = False
        for ticket in tickets:
            if ticket[2] == "Sold":
```

```
            print(f"{ticket[0]:<15}{ticket[1]:<15}")
            sold_found = True
    if not sold_found:
        print("No tickets have been sold yet.")

# Task 5: Calculate Remaining Tickets (Includes Bonus stats)
elif choice == "5":
    total_tickets = 0
    sold_tickets = 0
    available_tickets = 0

    # Loop through list to count based on conditions
    for ticket in tickets:
        total_tickets += 1
        if ticket[2] == "Sold":
            sold_tickets += 1
        elif ticket[2] == "Available":
            available_tickets += 1

    print("\n--- Remaining Tickets Summary ---")
    print(f"Total Tickets: {total_tickets}")
    print(f"Sold Tickets: {sold_tickets}")
    print(f"Available Tickets: {available_tickets}")

    # Bonus: Percentage of tickets sold
    if total_tickets > 0:
        percentage_sold = (sold_tickets / total_tickets) * 100
        print(f"Percentage Sold: {percentage_sold:.1f}%")

# Exit Program
elif choice == "6":
    print("\nThank you for using the Cricket Ticket System. Goodbye!\n")
    break

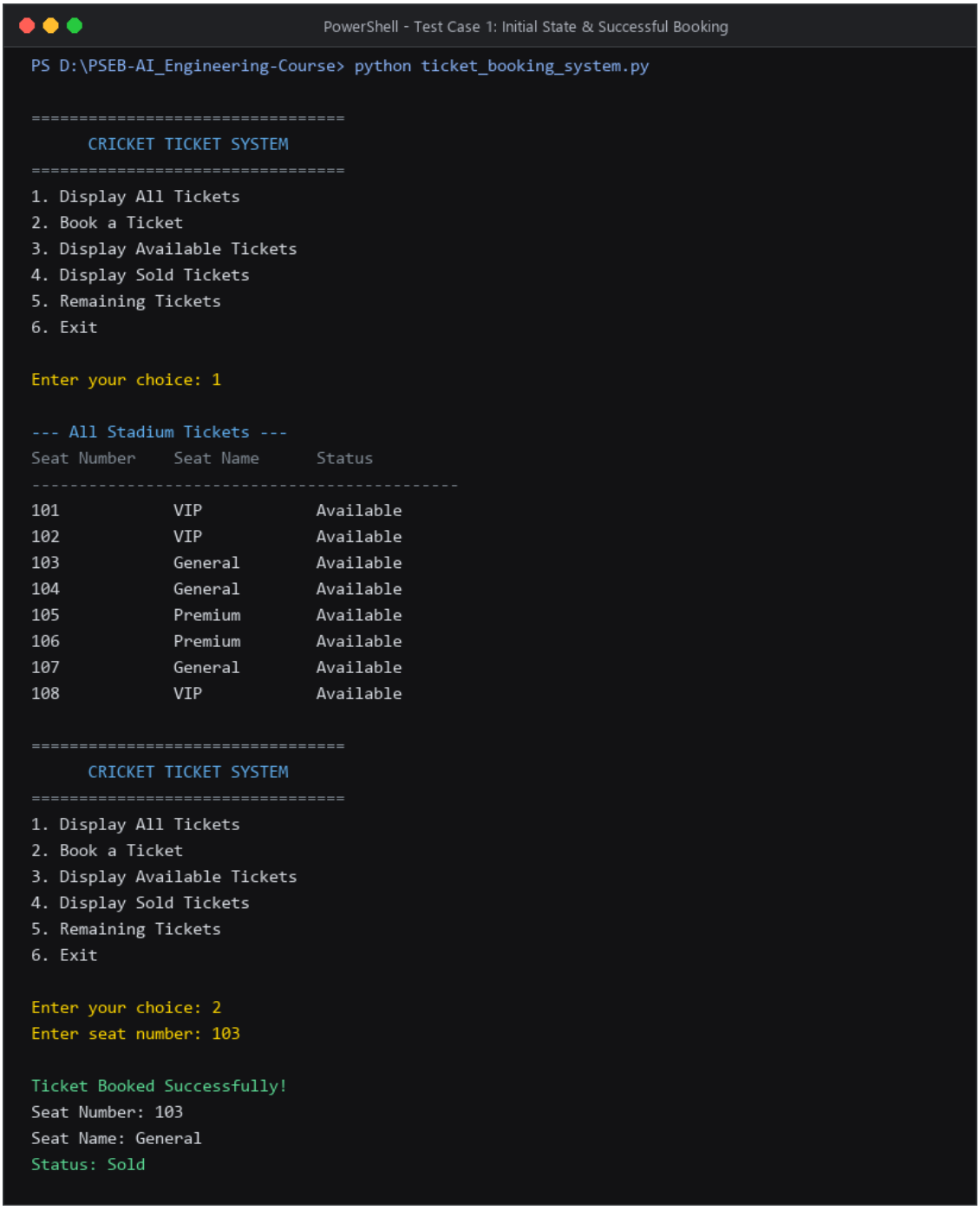
# Invalid menu selection
else:
    print("\nInvalid option! Please enter a number from 1 to 6.")
```

4. Test Cases & Execution Screenshots

In accordance with submission requirements, below are the designated test scenarios demonstrating successful booking, duplicate booking prevention, category filtering, and remaining capacity calculation.

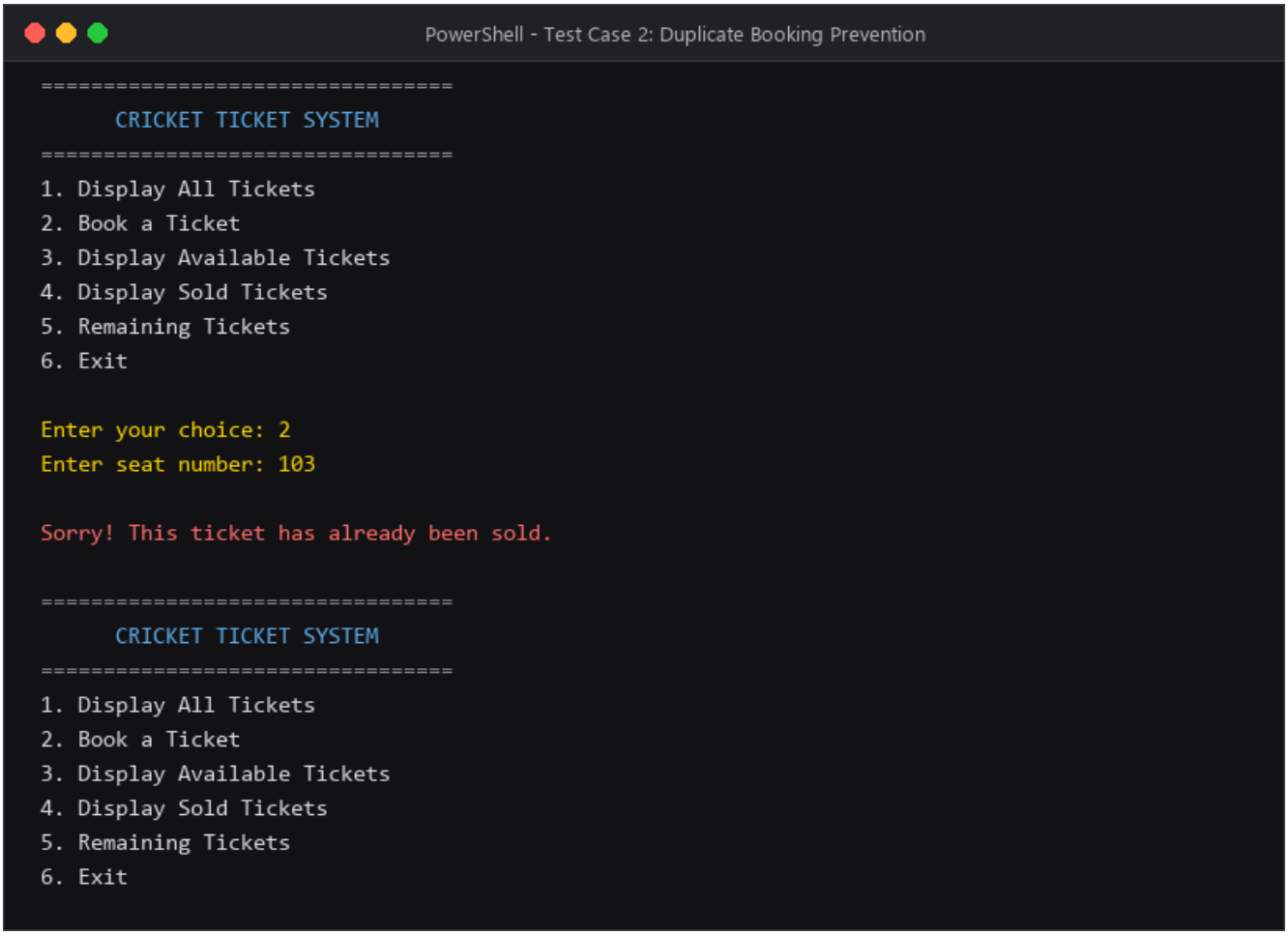
Test Case 1: Initial State & Successful Ticket Booking

- Operation: Display all tickets (Option 1), then book available seat number 103 (Option 2).
- Expected Result: Status of seat 103 updates in-place to 'Sold' with confirmation details.



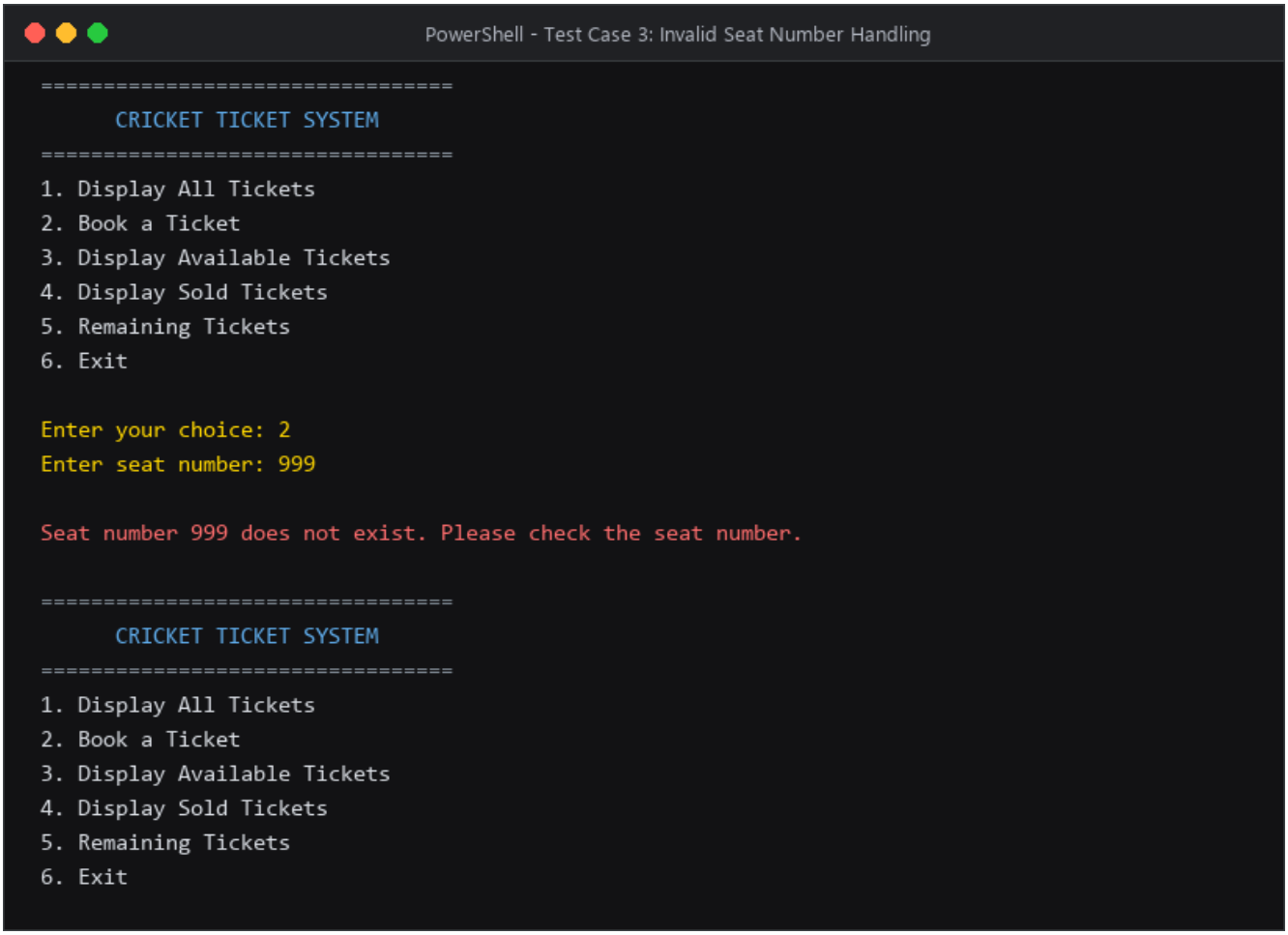
Test Case 2: Prevention of Duplicate Booking

- Operation: Attempting to purchase seat number 103 a second time (Option 2).
- Expected Result: System rejects booking and displays 'Sorry! This ticket has already been sold.'



Test Case 3: Invalid Seat Number & Input Handling

- Operation: Entering an invalid or non-existent seat number (e.g. 999).
- Expected Result: System outputs an error stating the seat does not exist.



Test Case 4: Category Filtering & Remaining Capacity Summary (Bonus)

- Operation: Display Available Tickets (Option 3), Sold Tickets (Option 4), and Remaining Tickets (Option 5).
- Expected Result: Displays accurate available/sold counts and percentage of capacity sold.

